DevOps Lab Experiment – 12

Build and Deploy a Complete DevOps Pipeline, Discussion on Best Practices and Q&A.

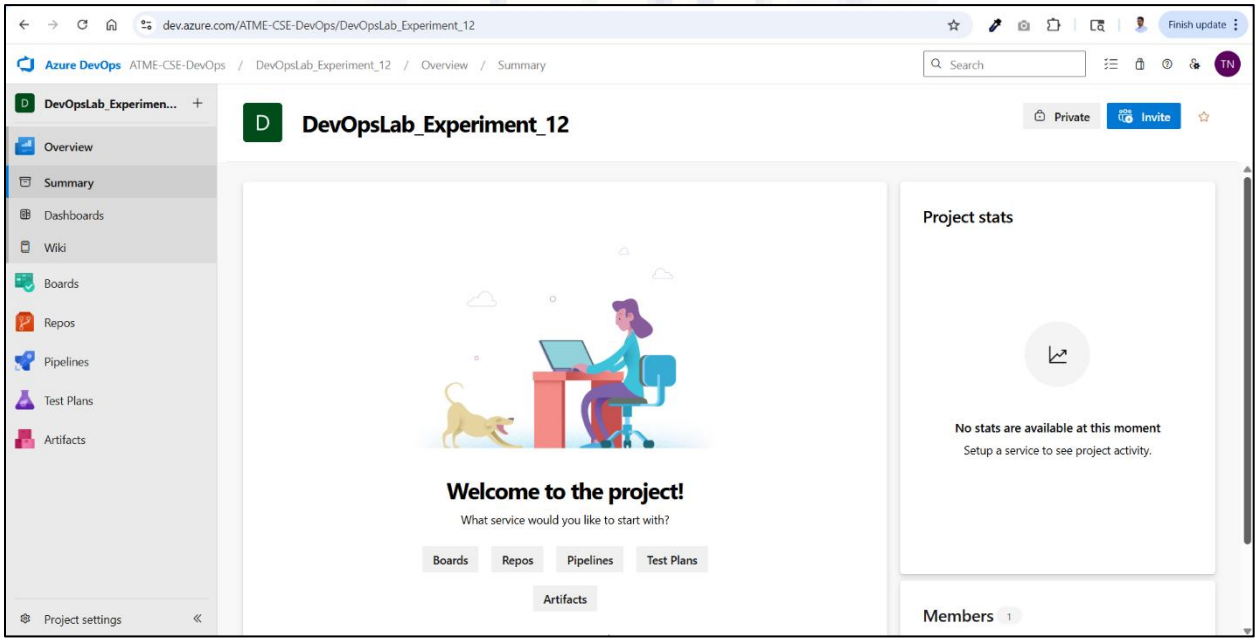
The **Build Pipeline (Continuous Integration)** is responsible for compiling the source code, executing tests, and producing artifacts.

The **Release Pipeline (Continuous Deployment)** utilizes these artifacts to deploy the application to a target environment.

Together, they enable an efficient and automated software delivery process.

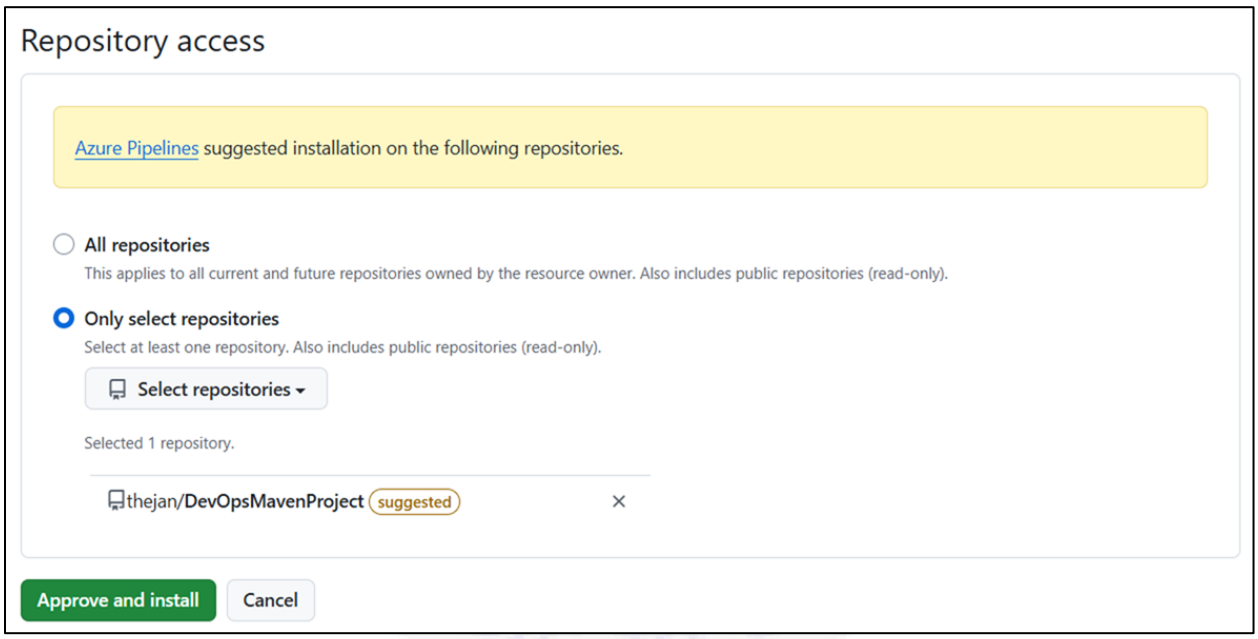
Build Pipeline (CI) Setup

Step 1 : Navigate to the Azure DevOps portal (https://dev.azure.com/) , login with your credentials, select the organization, create a new project (Ex. *DevOpsLab_Experiment-12*) and select the newly created project.

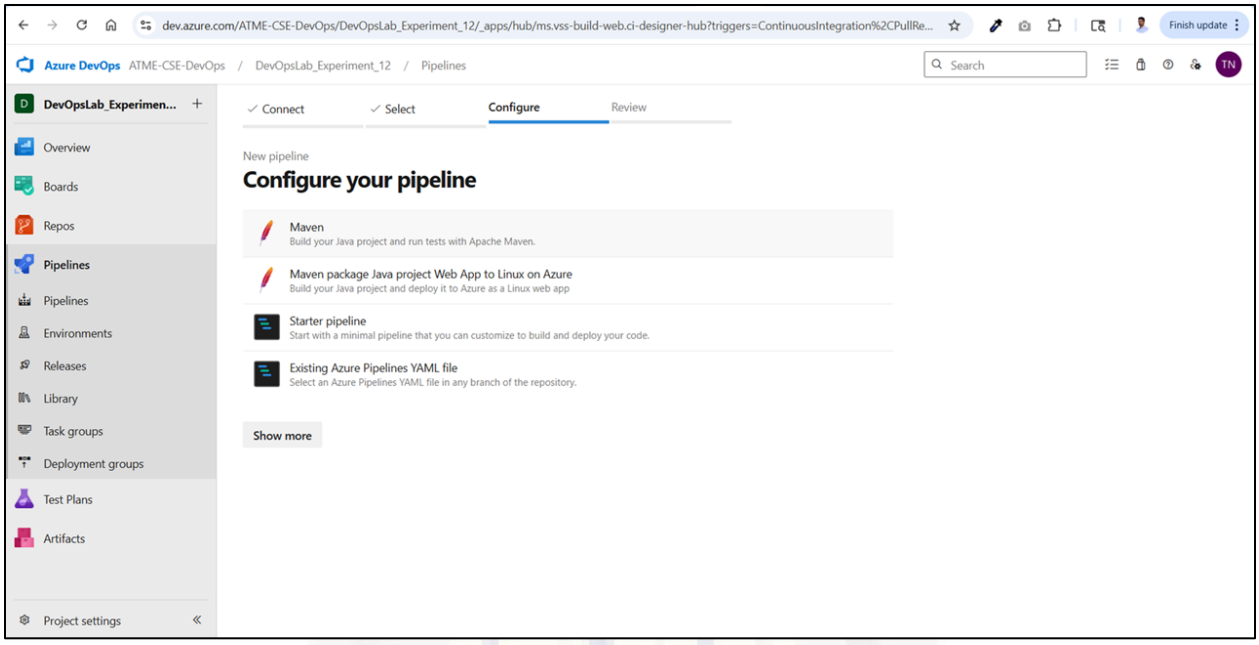


Step 2 : Create a build pipeline

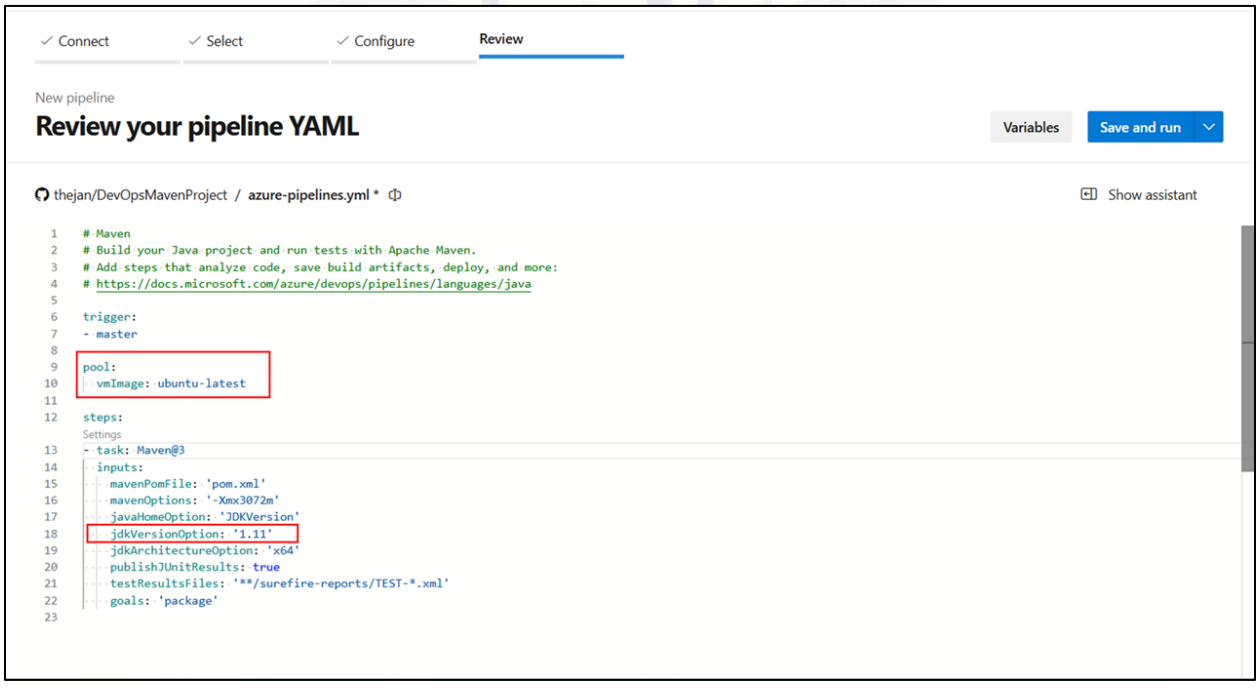
- Navigate to Pipelines → Select Pipelines → Click on **Create Pipeline**
- Select GitHub → Select your repository (Maven/Gradle project)
- Grant access to the Git repository by entering the verification code received via email.



- In the configuration step, select **Maven** to build the Java project using Apache Maven.



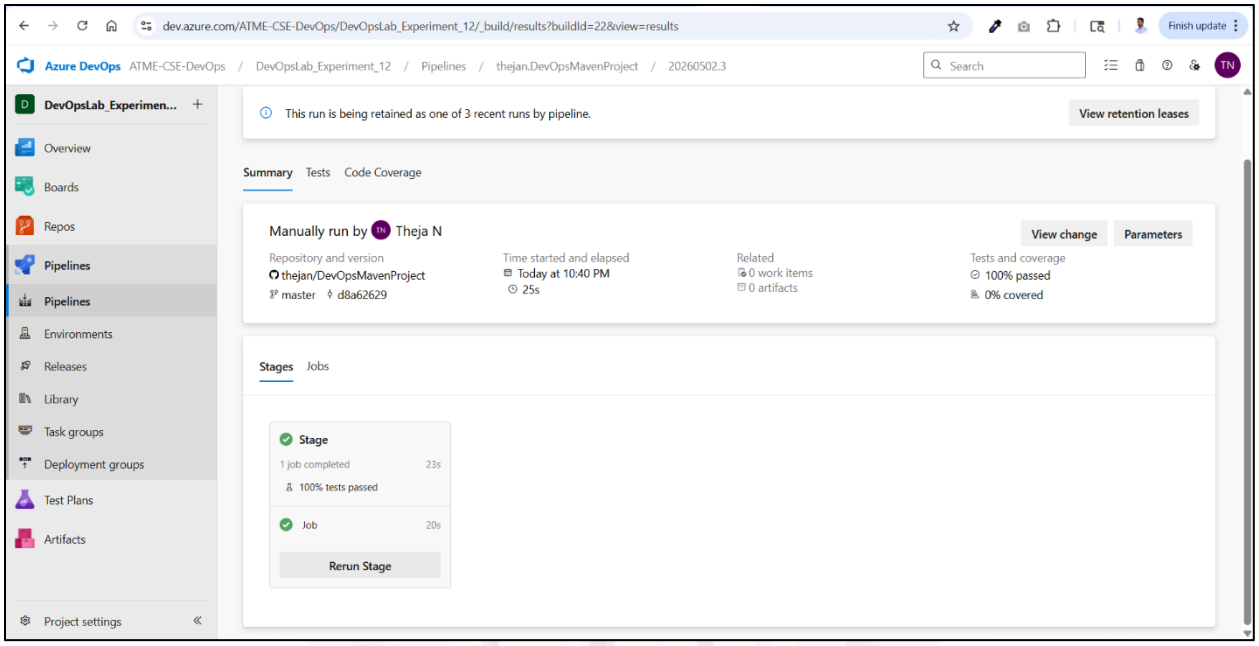
- Review the YAML file, click on **Save and run**



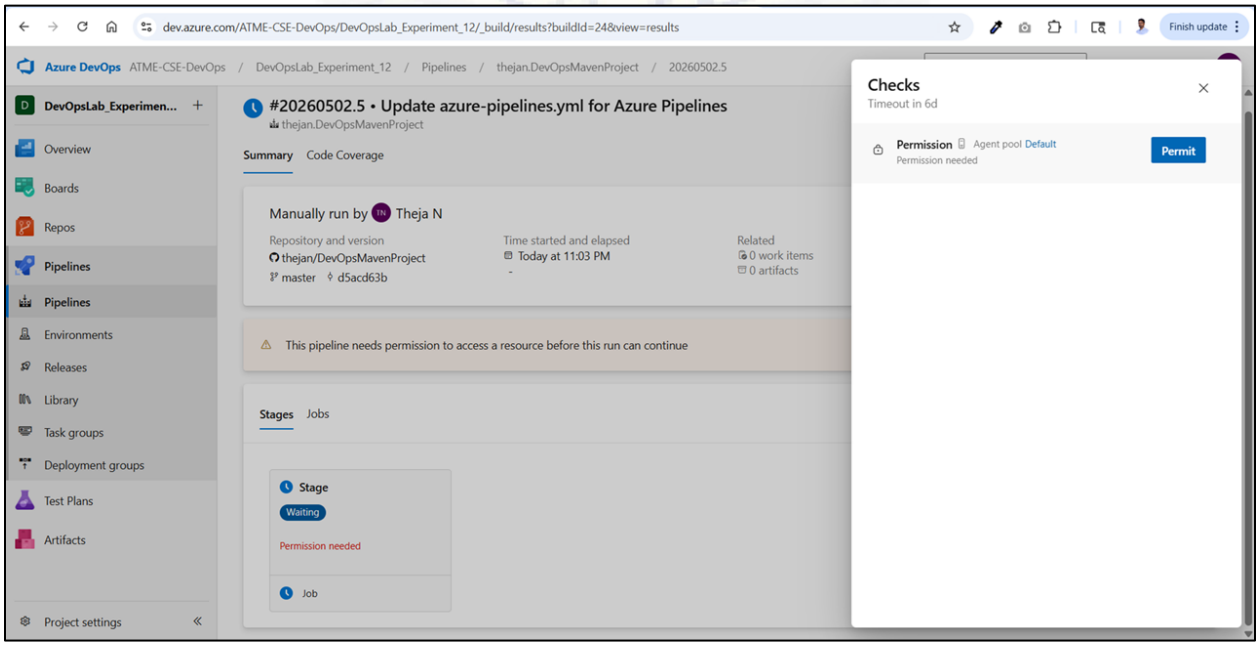
The pipeline will run on a **Microsoft-hosted agent** using the latest Ubuntu operating system image.

If a self-hosted agent is configured on the local machine, update the pipeline configuration as follows (YAML file)

```
trigger:
- master

pool:
  name: Default

steps:
- task: Maven@3
  inputs:
    mavenPomFile: 'pom.xml'
    mavenOptions: '-Xmx3072m'
    javaHomeOption: 'JDKVersion'
    jdkVersionOption: '1.25'
    jdkArchitectureOption: 'x64'
    publishJUnitResults: true
    testResultsFiles: '**/surefire-reports/TEST-*.xml'
    goals: 'package'

# Copy JAR to artifact staging
- task: CopyFiles@2
  inputs:
    Contents: '**/*.jar'
    TargetFolder: '$(Build.ArtifactStagingDirectory)'

# Publish artifact
- task: PublishBuildArtifacts@1
  inputs:
    pathToPublish: '$(Build.ArtifactStagingDirectory)'
    artifactName: 'drop'
```

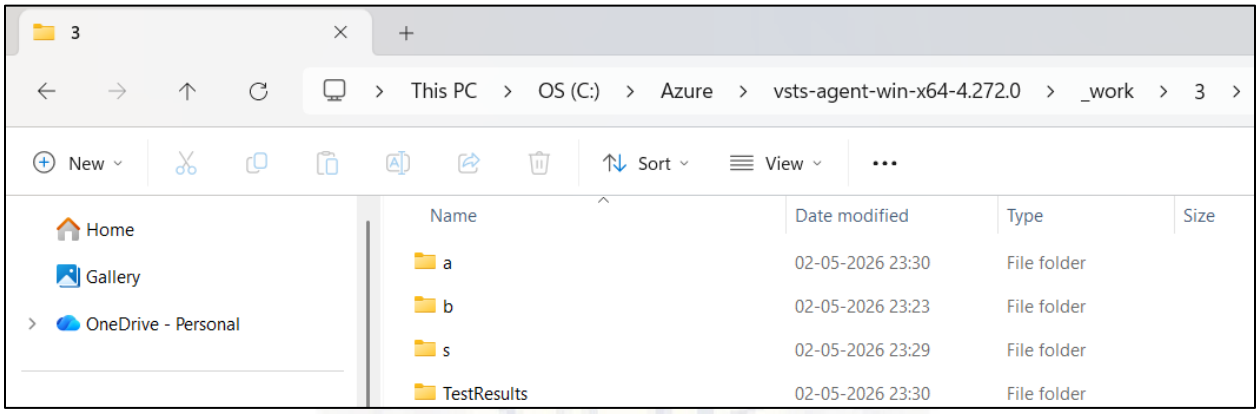


- If the pipeline execution is paused with a **Permission needed** message, grant the required permissions to allow access to the resource. Click on **Permit**.



- Verify that the job has been executed successfully on the locally configured (self-hosted) agent

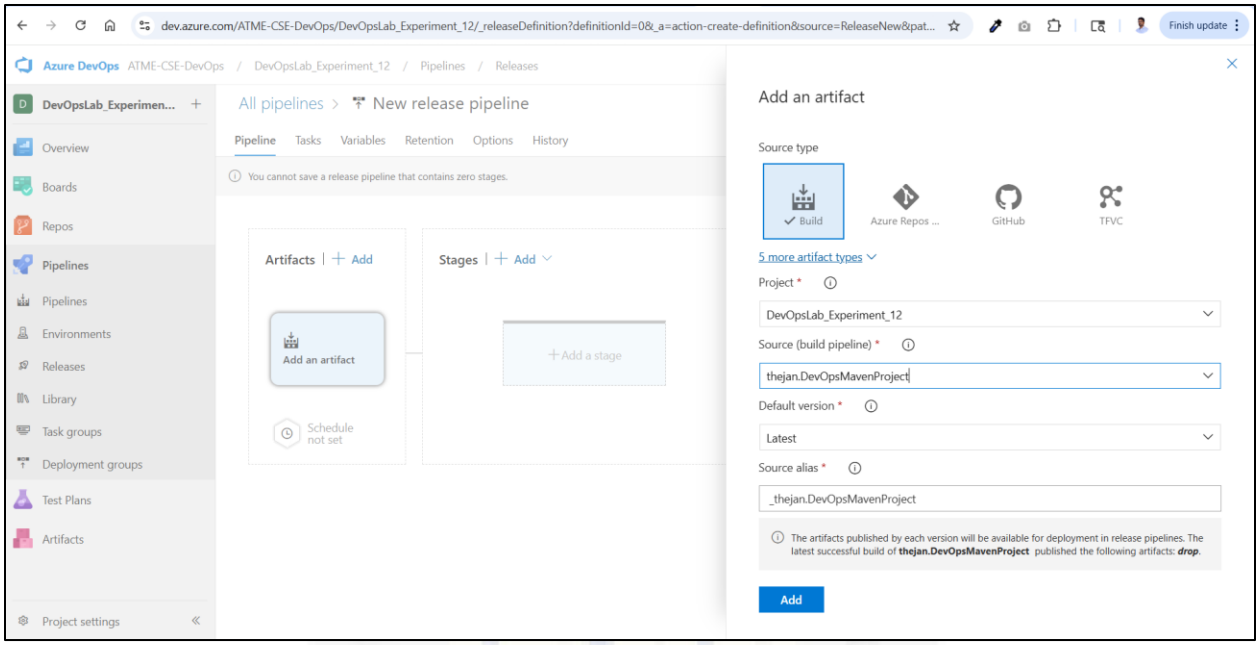
- Navigate to the working directory of the agent and inspect the folders inside the latest build directory



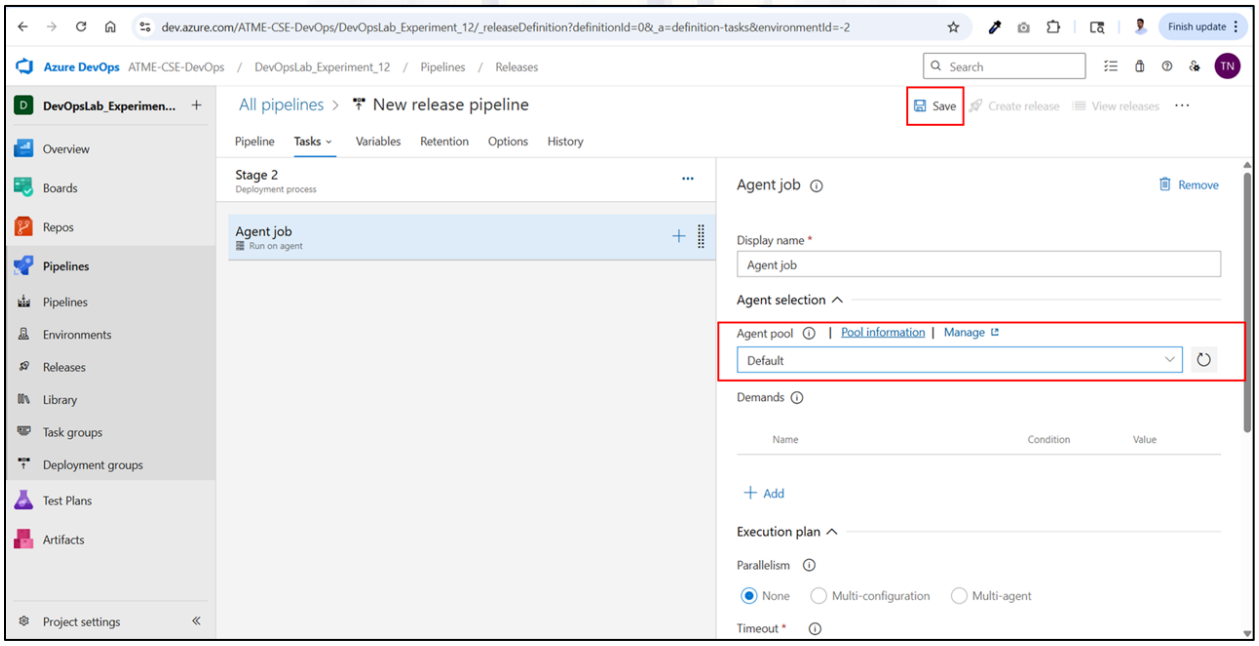
Release Pipeline (CD) Setup

Step 3 : Create a release pipeline

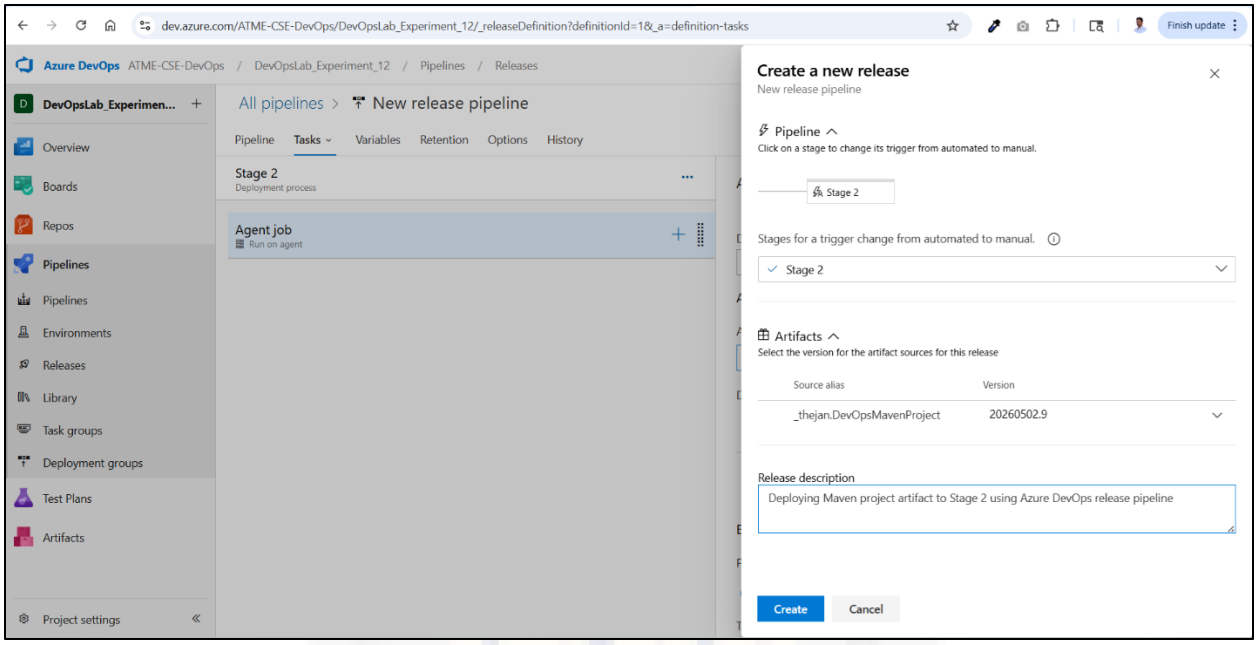
- Go to your project. Click Pipelines → Releases → New pipeline
- Click Add an artifact and select the **build** Source type



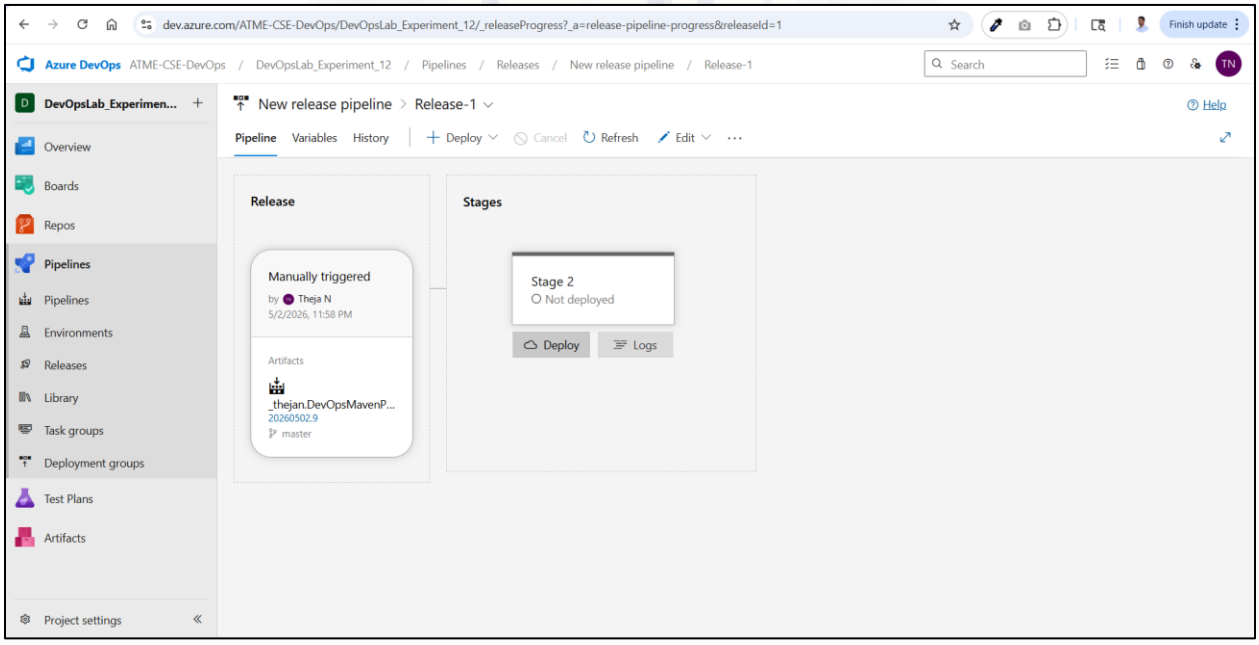
- Add a stage and configure the Agent job. Select the **Default** Agent pool.



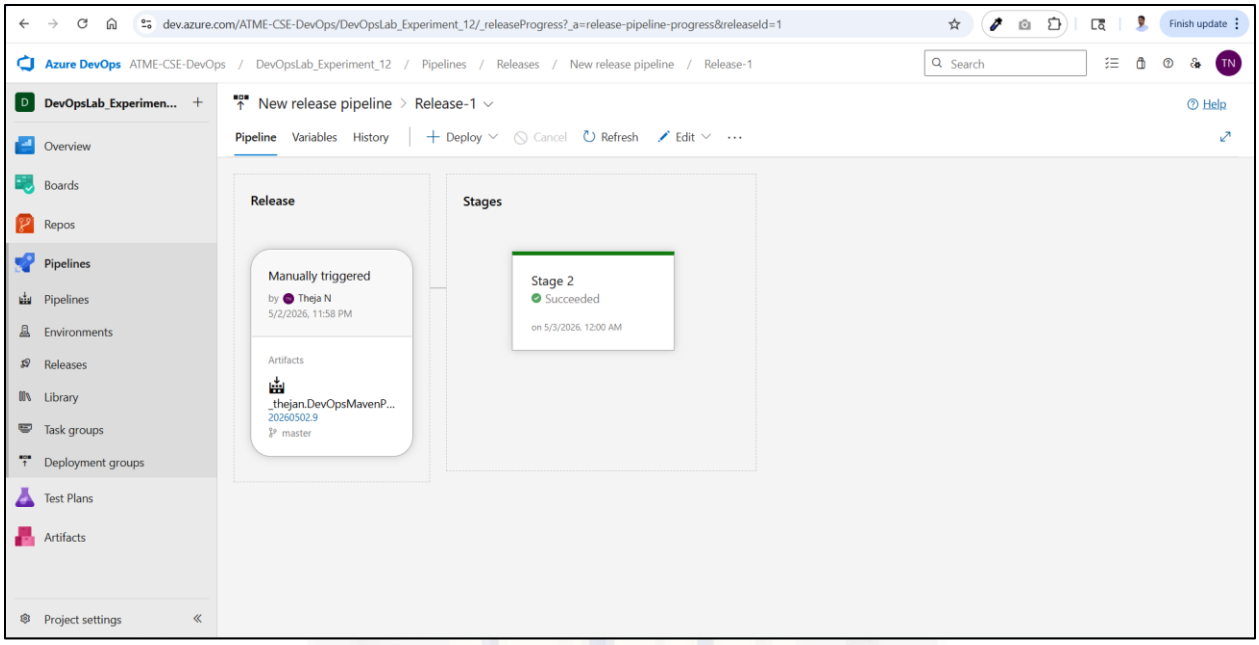
- Create a release by selecting the Stage, Artifact and entering a Release description



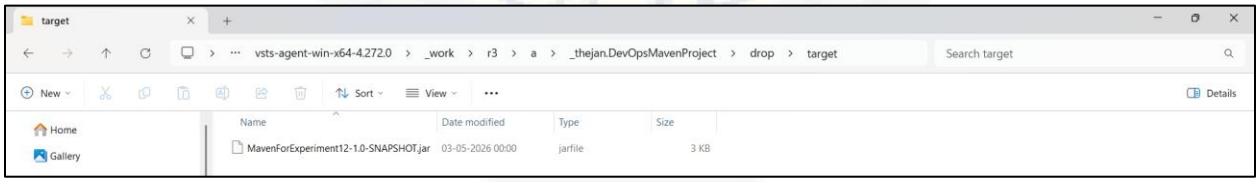
- View releases → select the created release → select the stage → click **Deploy**



- Monitor the deployment and verify that deployment status is succeeded



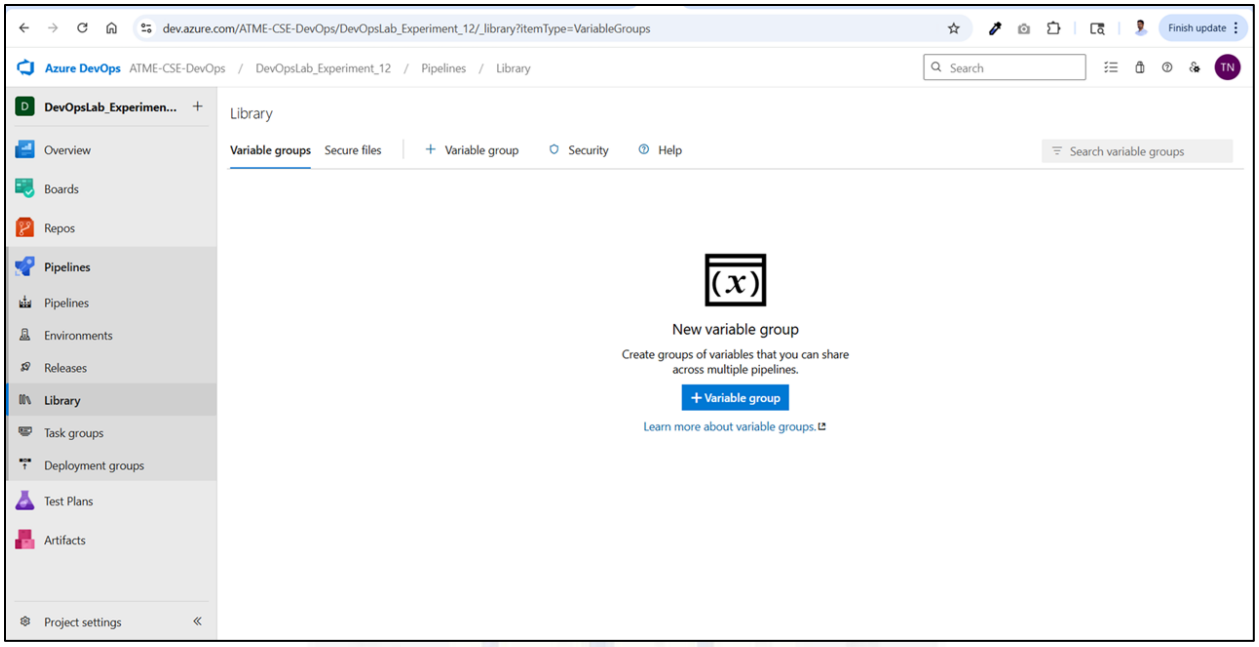
- Verify that the artifact has been successfully deployed inside the latest release folder in the agent's working directory.



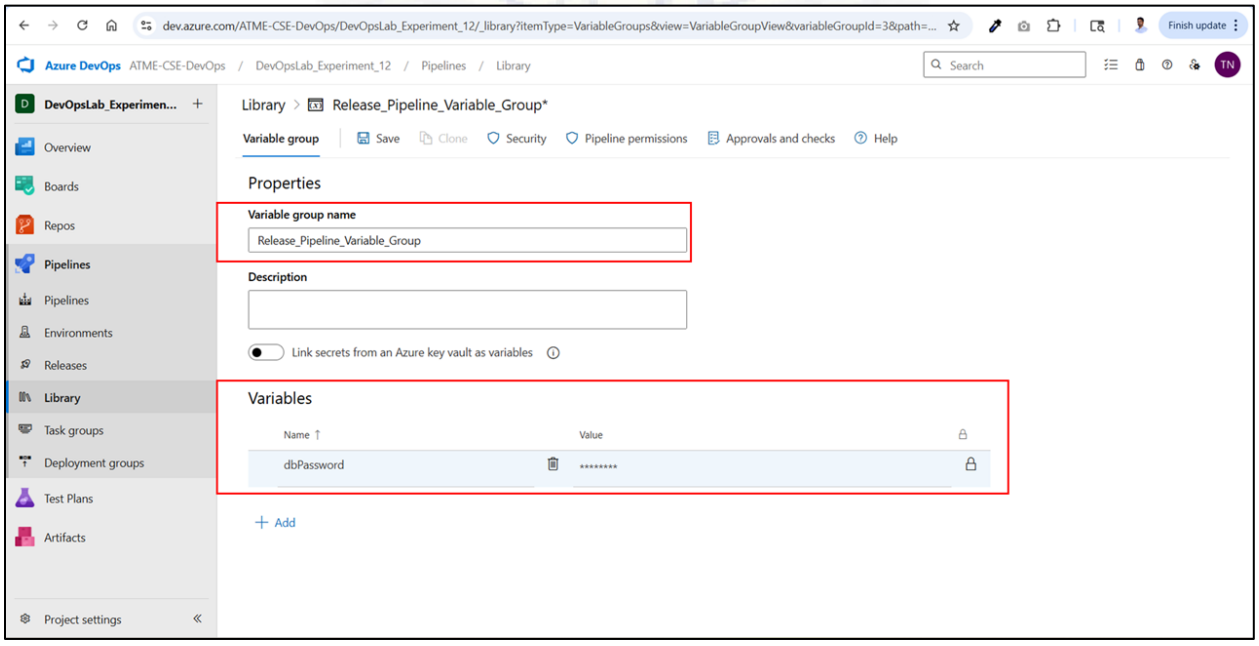
A release pipeline automates deployment by taking **build artifacts** and deploying them to a **target environment** using defined stages.

Simulating Secrets with Variable Groups

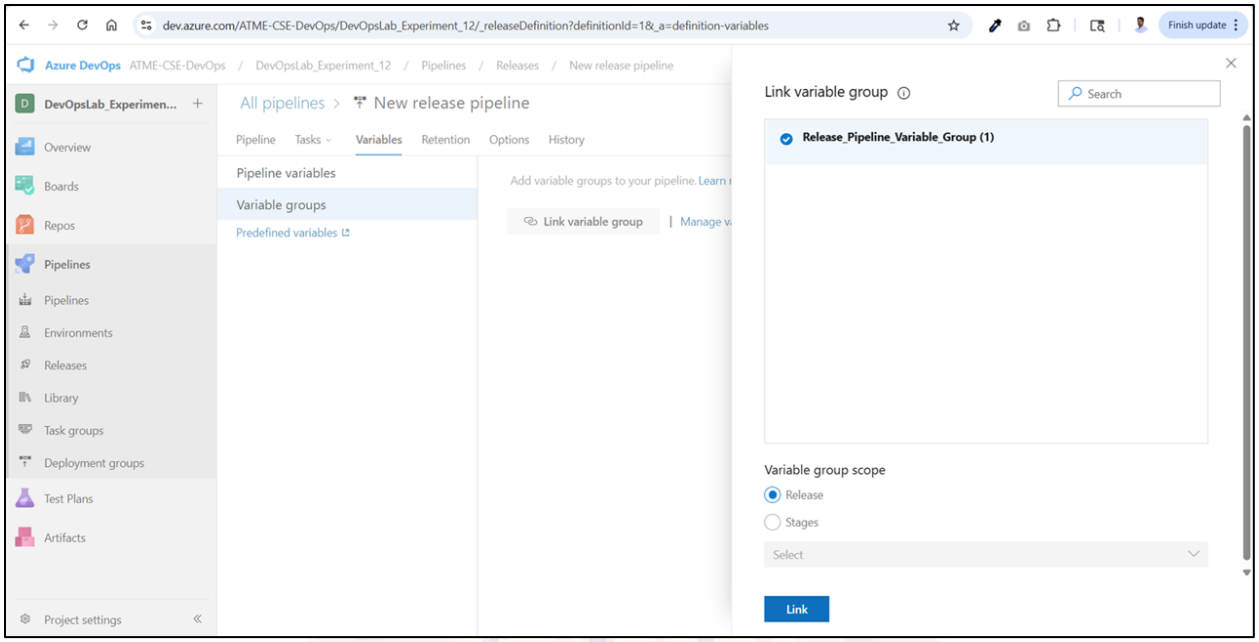
Step 4 : Go Pipelines → Library → click on **+Variable group**



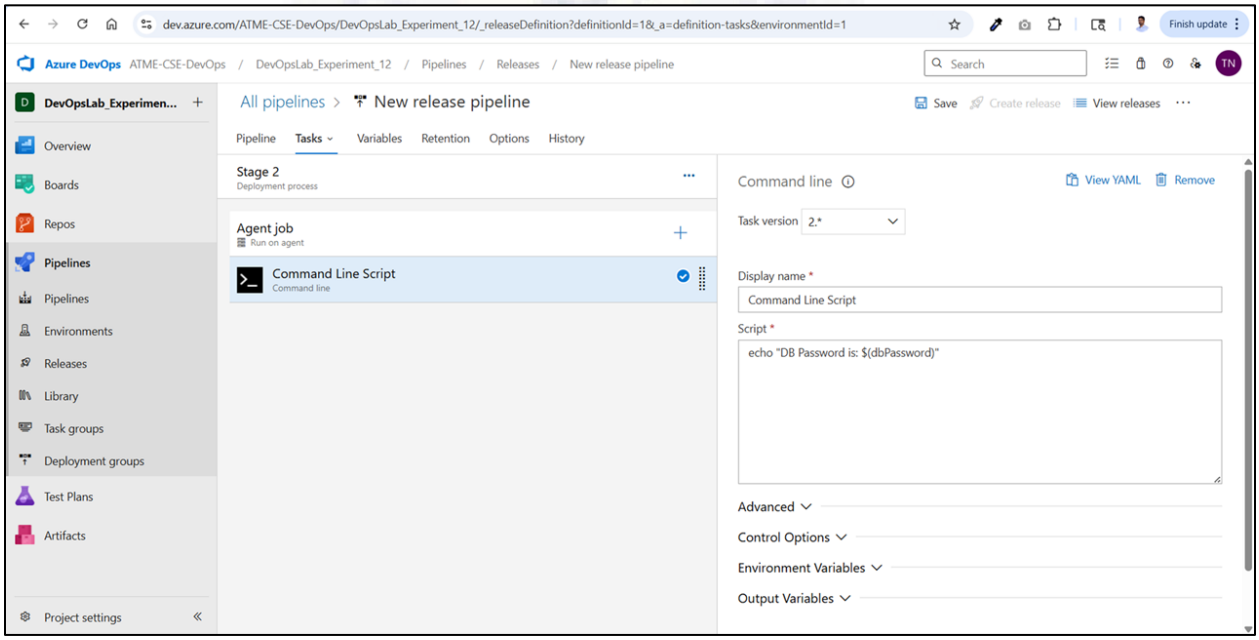
- Type Variable group name, Variable name, its value, set the variable as a secret variable and then click **Save**



- Link the Variable group with the pipeline



- Edit the release pipeline, go to the Tasks section, add a Command Line task under the Agent job, enter echo "DB Password is: $(dbPassword)" in the script, and save the pipeline.



- Run the release pipeline, download the logs, and verify that the dbPassword variable has been echoed.

Discussion on Best Practices (Examples)

- Use version control (e.g., Git) to manage all YAML and pipeline configuration files.
- Simulate builds or deployments (using echo commands) when agents or resources are unavailable.
- Store sensitive data such as passwords, API keys, and tokens securely using **variable groups** or **Azure Key Vault**.
- Structure pipelines into clear stages such as **Build → Test → Deploy** for better readability and control.
- Regularly review pipeline logs to identify and troubleshoot errors effectively.
- Test the application locally before triggering the pipeline to reduce build failures.
- Use appropriate agent pools (Microsoft-hosted or self-hosted) based on project requirements.
- Keep pipeline tasks updated (e.g., use Maven@4 instead of deprecated versions).
- Avoid hardcoding values in scripts; use variables for flexibility and security.
- Grant only necessary permissions to pipelines and resources to maintain security.
- Maintain proper naming conventions for pipelines, stages, and artifacts for better organization.
- Enable continuous integration (CI) triggers to automatically build on code changes.
- Use artifacts efficiently to pass build outputs between stages.

VIVA QUESTIONS

1. What is the difference between **Continuous Integration (CI)** and **Continuous Deployment (CD)**?
2. What is the purpose of a **build pipeline** in Azure DevOps?
3. What is a **release pipeline**, and how does it differ from a build pipeline?
4. What are **artifacts**, and why are they important in DevOps pipelines?
5. What is the role of an **agent** in Azure DevOps?
6. What is the difference between a **Microsoft-hosted agent** and a **self-hosted agent**?
7. What is a **YAML pipeline**, and why is it preferred?
8. What is the purpose of the pool keyword in YAML?
9. Why do we use **Maven** in the build pipeline?
10. What is a **variable group**, and how is it used in pipelines?

11. What is a **secret variable**, and why is it important?
12. Why are secret variables masked in pipeline logs?
13. What is meant by **pipeline permissions**, and why are they required?
14. What are the different **stages in a pipeline**?
15. What are some **best practices in DevOps pipelines**?